

Bridging the Gap: Enabling Natural Language Queries for NoSQL Databases through Text-to-NoSQL Translation

Jinwei Lu¹, Yuanfeng Song², Zhiqian Qin¹, Haodi Zhang³, Chen Zhang¹, Raymond Chi-Wing Wong⁴

¹The Hong Kong Polytechnic University, Hong Kong, China

²WeBank Co., Ltd, Shenzhen, China ³Shenzhen University, Shenzhen, China

⁴The Hong Kong University of Science and Technology, Hong Kong, China

ABSTRACT

NoSQL databases have become increasingly popular due to their outstanding performance in handling large-scale, unstructured, and semi-structured data, highlighting the need for user-friendly interfaces to bridge the gap between non-technical users and complex database queries. In this paper, we introduce the Text-to-NoSQL task, which aims to convert natural language queries into NoSQL queries, thereby lowering the technical barrier for non-expert users. To promote research in this area, we developed a novel automated dataset construction process and released a large-scale and open-source dataset for this task, named TEND (short for Text-to-NoSQL Dataset). Additionally, we designed a SLM (Small Language Model)-assisted and RAG (Retrieval-augmented Generation)-assisted multi-step framework called SMART, which is specifically designed for Text-to-NoSQL conversion. To ensure comprehensive evaluation of the models, we also introduced a detailed set of metrics that assess the model’s performance from both the query itself and its execution results. Our experimental results demonstrate the effectiveness of our approach and establish a benchmark for future research in this emerging field. We believe that our contributions will pave the way for more accessible and intuitive interactions with NoSQL databases.

1 INTRODUCTION

In recent years, with the rapid development of Internet and big data technologies, NoSQL databases have garnered increasing attention due to their outstanding performance in handling large-scale, unstructured, and semi-structured data. NoSQL databases offer high scalability, high performance, and flexible data models, effectively addressing the challenges posed by massive data volumes and high-concurrency access. Within the database and data mining community, NoSQL databases have also been the subject of extensive research and have gained widespread recognition [4, 6, 9, 11, 14, 14, 22, 41].

Despite the numerous advantages of NoSQL databases, the need for specialized knowledge to write query statements and manage data effectively presents a high technical barrier for users. The query languages and operational methods of NoSQL databases differ significantly from traditional SQL, posing considerable difficulties for non-expert users. Therefore, bridging the gap between natural language (NL) and NoSQL query syntax is a crucial step towards further promoting these powerful database systems.

To address this challenge, we introduce the task of **Text-to-NoSQL** for the first time in this paper. This new paradigm aims to translate natural language queries (NLQs) into corresponding NoSQL queries, thereby significantly reducing the technical difficulty of using NoSQL databases (as shown in Figure 1). Its goal is to facilitate more intuitive interactions between users and NoSQL databases, enabling a broader audience, including those without extensive programming backgrounds, to benefit from these systems.

Furthermore, recognizing the critical role of benchmark datasets in driving research and development, we present **TEND**, a large-scale and open-source Text-to-NoSQL task dataset. TEND is designed to provide a comprehensive resource for researchers and developers working on natural language to query translation systems, particularly those focused on NoSQL environments. It offers a standardized evaluation platform, promoting comparison between different approaches, accelerating innovation, and fostering competition within the Text-to-NoSQL community.

To address the significant human and time costs associated with constructing large-scale datasets, we propose a novel semi-automated process for constructing a large-scale, high-quality Text-to-NoSQL dataset. Specifically, the Text-to-NoSQL dataset consists of three components: **databases, NoSQL queries, and corresponding natural language queries (NLQs)**. First, as described in Section 2.2, **we employ a graph traversal algorithm to organize relational database tables into distinct clusters based on foreign key relationships**. Using the Depth-First Search algorithm, we transform these foreign key relationships into nested structures suitable for NoSQL databases, ultimately constructing a comprehensive NoSQL database. **Next, a sophisticated process combining large language models (LLMs) and code programs is utilized to generate and refine NoSQL queries**. Finally, multiple LLMs are employed to expand the NLQs in the dataset. All LLM-based processes in the pipeline leverage advanced Chain-of-Thoughts (CoT) techniques, generated by state-of-the-art LLMs such as GPT-4o, to enhance the reasoning capabilities of second-tier LLMs (e.g., GPT-4o-mini). For additional details, please refer to Section 2. This dataset construction approach not only significantly reduces manual intervention but also ensures data consistency and accuracy.

To advance research on the Text-to-NoSQL task within the data mining and database communities, **we designed a SLM (Small Language Model)-assisted and RAG (Retrieval-augmented Generation)-assisted multi-step framework called SMART**. SMART is divided into four main processes: (1) *SLM-based Query Generation*, (2) *SLM-based Query Generation*, (3) *Predicted Schema-driven and Retrieved*

The partial TEND dataset and online technical report with more experimental results and analysis are available at https://1drv.ms/f/c/80e862c5ba9a0a46/EiLzN6bnOyhFnd66dl0tEOQB9T_YTVzy9L_7Bp8kHpb02g?e=ntl9eE

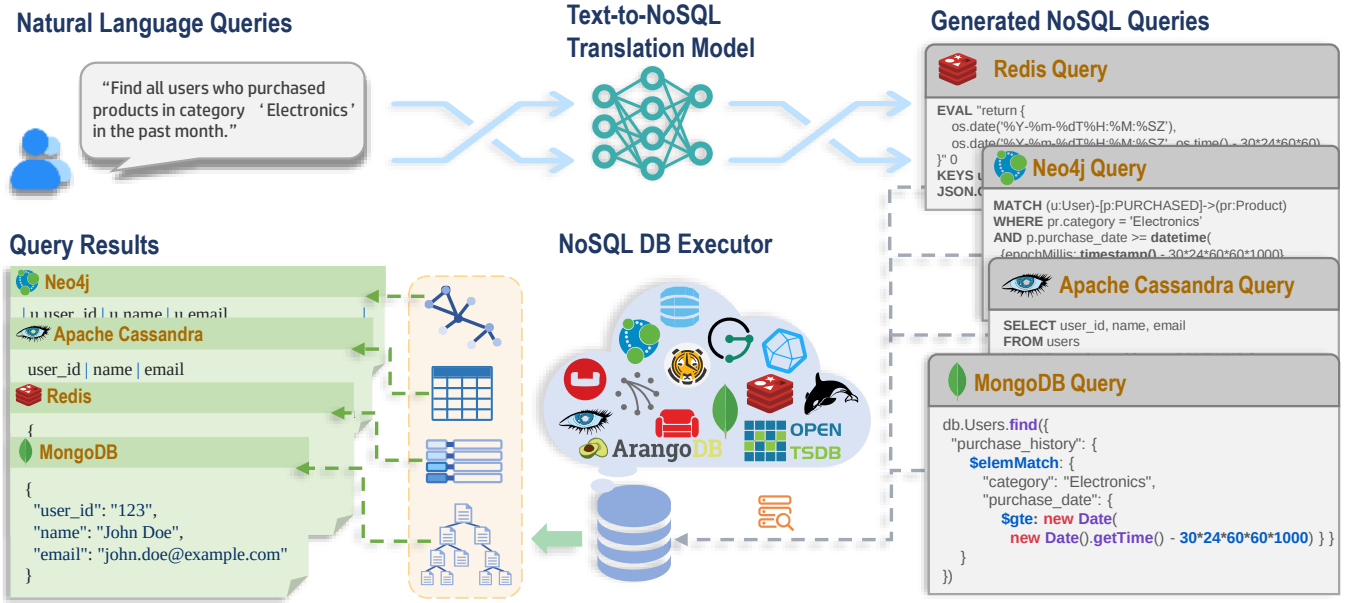


Figure 1: An example of Text-to-NoSQL involves a Text-to-NoSQL model converting a user’s natural language query into a NoSQL query, which is then executed in the corresponding NoSQL database to obtain the desired result.

Example-driven Query Refinement, and (4) Execution Result-based Query Optimization. Specifically, we have the following. (i) Establishing the mapping relationship between NLQ and database schemas is a complex task. If the model is required to complete the conversion from NLQ to NoSQL query directly, it becomes more challenging for the model to establish this mapping relationship effectively. To address this issue, we adopted a step-by-step approach, fine-tuning schema predictors to predict this mapping relationship. For example, We fine-tuned various SLMs to predict the database fields used in queries, the non-database fields named in queries, as well as the fields in the final execution result documents. (ii) After fine-tuning, SLMs are generally better aligned with the preferences of the dataset without the need for manual summarization of these preferences and additional declarations through prompt engineering. Leveraging this characteristic, we fine-tuned a Text-to-NoSQL SLM to directly transform NLQs into initial NoSQL queries based on the database schema. (iii) After obtaining the schemas predicted by the SLM and the initial NoSQL query, we retrieve examples from the **TEND training set** based on this predicted information and the current NLQ. Specifically, we calculate the cosine similarity of the predicted schemas, NoSQL query, and NLQ with various elements in the training set examples, obtaining the top-k most relevant examples as reference examples in the RAG prompt. The LLM then refines the NoSQL query predicted by the SLM based on these examples. (iv) Similar to the query refiner, the query optimizer adopts the same retrieval strategy to retrieve the top-k most relevant examples. The difference is that in the query optimizer, the retrieved queries and the adjusted queries are executed to obtain result documents. These documents serve as the basis for the LLM to further optimize the query.

To provide a comprehensive evaluation of the performance of Text-to-NoSQL models, we specifically designed evaluation metrics for this task. The metrics are divided into two main categories, focusing on the query itself and its execution results. Metrics for the query itself can assess the model’s understanding of the logical structure of the query statements, while metrics for the execution results can better evaluate the model’s performance in real-world scenarios. Specifically, we use **Exact Match** (EM, which focuses on the exact match between the generated query and the gold query) as the primary evaluation metric for the query itself. Additionally, we refined two further metrics: **Query Stages Match** (QSM, which focuses on the match between the generated query and the gold query in the query stage) and **Query Fields Coverage** (QFC, which focuses on the match between the fields used in the generated query and the gold query) to provide a more detailed evaluation of the query itself. For the evaluation of execution results, we use **Execution Accuracy** (EX, which focuses on the exact match of the execution results) as the primary evaluation metric. We also designed **Execution Fields Match** (EFM, which focuses on the fields in the execution results) and **Execution Value Match** (EVM, which focuses on the values in the execution results) as further detailed evaluation metrics.

Finally, we validated the feasibility and complexity of the Text-to-NoSQL task through extensive experiments. Basic models such as Seq2Seq and Transformer, due to their simplistic architectures, were unable to learn the complex syntax of NoSQL during training and thus failed to achieve Text-to-NoSQL predictions, highlighting the intricacy of the task. In contrast, LLMs were augmented solely with the RAG technology and the Fine-tuned Llama, which was directly fine-tuned on the TEND training set, achieved execution accuracies of 52.76% and 53.12% on the TEND test set, respectively.

Moreover, SMART attained an execution accuracy of 65.08%, which amply demonstrates the feasibility of the Text-to-NoSQL task.

To sum up, the primary contributions of this paper can be summarized as follows:

- We are the first to propose the Text-to-NoSQL task, which aims to reduce the technical difficulty of using NoSQL databases by automatically translating NLQs into corresponding NoSQL queries, thereby bridging the gap between NL and NoSQL queries.
- We introduced TEND, a large-scale and open-source benchmark dataset for the Text-to-NoSQL task, with the aim of providing a standardized evaluation platform to attract more researchers to contribute to the development of the Text-to-NoSQL community.
- We propose a novel large-scale dataset construction process and aim at reducing the time and labor costs for researchers while ensuring the consistency and accuracy of the data in the dataset.
- We designed a sophisticated multi-step framework, SMART, which cleverly combines SLM and RAG technologies to effectively achieve Text-to-NoSQL task predictions, providing a model baseline for future work on this task.
- We designed specialized evaluation metrics for the Text-to-NoSQL model. By refining the evaluation of the query itself and its execution results, we can comprehensively analyze the model’s ability to understand the logic of the query and its generalization capability in real-world scenarios. The aim is to foster healthy competition among models within the Text-to-NoSQL community, thereby promoting the development of this community.
- We validated the complexity and feasibility of the Text-to-NoSQL task through extensive experiments.

The rest of this paper is structured as follows. Section 2 details the process of constructing the Text-to-NoSQL dataset. Section 3 explains the specific setup for dataset creation and provides statistics about the dataset. Section 4 discusses the design and implementation of SMART. Section 5 outlines the experimental setup and analyzes the results. Section 6 reviews related studies in the field. Finally, Section 7 summarizes the key contributions of this paper.

2 THE PROPOSED PIPELINE FOR CONSTRUCTING THE TEXT-TO-NOSQL BENCHMARK

In this section, we provide a detailed explanation of the construction pipeline of TEND.

2.1 Overview

The construction process of TEND is a semi-automatic pipeline that can convert a Text-to-SQL dataset into a Text-to-NoSQL dataset. Its automation is mainly divided into three steps: Database Transformation (Section 2.2), Advanced CoT-driven Query Transformation (Section 2.3), and Question Extension by Multi-LLM (Section 2.4) (as shown in Figure 2). Finally, we manually reviewed and adjusted the entire dataset, removing examples that did not meet expectations

Algorithm 1: The Pipeline for Database Transformation

```

1 Input: SQL Database  $D_{SQL}$ 
2 Output: NoSQL Database  $D_{NoSQL}$ 
3 begin
4   // Relational Table Grouping
5    $C_{SQL} \leftarrow \{\}$ 
6   for each  $t \in D_{SQL}$  do
7     // Grouping by Foreign Key Relationships
8     for each  $t' \in \text{GetFKTableFrom}(t)$  do
9       if  $\text{ExistTable}(t', C_{SQL})$  then
10        |  $\text{MergeC}(\text{GetC}(t', C_{SQL}), \text{GetC}(t, C_{SQL}))$ 
11      else
12        |  $\text{AddTable}(t', \text{GetC}(t, C_{SQL}))$ 
13    // Main Table Obtaining
14    for each  $c \in C_{SQL}$  do
15      |  $c[T_{main}] \leftarrow \text{GetMainTable}(c)$ 
16    // Recursive Data Transformation
17     $D_{NoSQL} \leftarrow []$ 
18    for each  $C \in C_{SQL}$  do
19      // Embed Main Table Data as Top-Level
20      // NoSQL Documents
21       $c \leftarrow \text{GetData}(C[T_{main}])$ 
22      // Recursively Join Tables via Foreign
23      // Keys
24       $Q \leftarrow T_{main}$ 
25      while  $Q \neq []$  do
26        |  $t \leftarrow Q.\text{pop}()$ 
27        | for each  $t' \in \text{GetFKTableTo}(t)$  do
28          |  $c[t] \leftarrow \text{GetData}(t', t[fk] \equiv t'[fk])$ 
29          |  $Q.\text{push}(t')$ 
30       $D_{NoSQL}.\text{append}(c)$ 
31  return  $D_{NoSQL}$ 

```

and standardizing programming conventions across the dataset. Below, we explain this process in detail.

2.2 Database Transformation

TEND consists of three parts: databases, NoSQL queries, and NLQs. The construction of databases is a labor-intensive and time-consuming task, involving extensive data collection and filtering. To reduce the substantial manpower and time required for database construction, we designed an algorithm to convert relational databases into NoSQL (MongoDB) databases, where we transformed the foreign key relationships between tables in the relational database into nested relationships in the NoSQL database. Below is a detailed explanation of the database transformation algorithm along with specific examples.

The algorithm for database transformation is shown in Algorithm 1, with a specific example illustrated in Figure 2(i). First, we traverse all tables and their potential foreign keys, grouping two tables connected through a foreign key into the same MongoDB

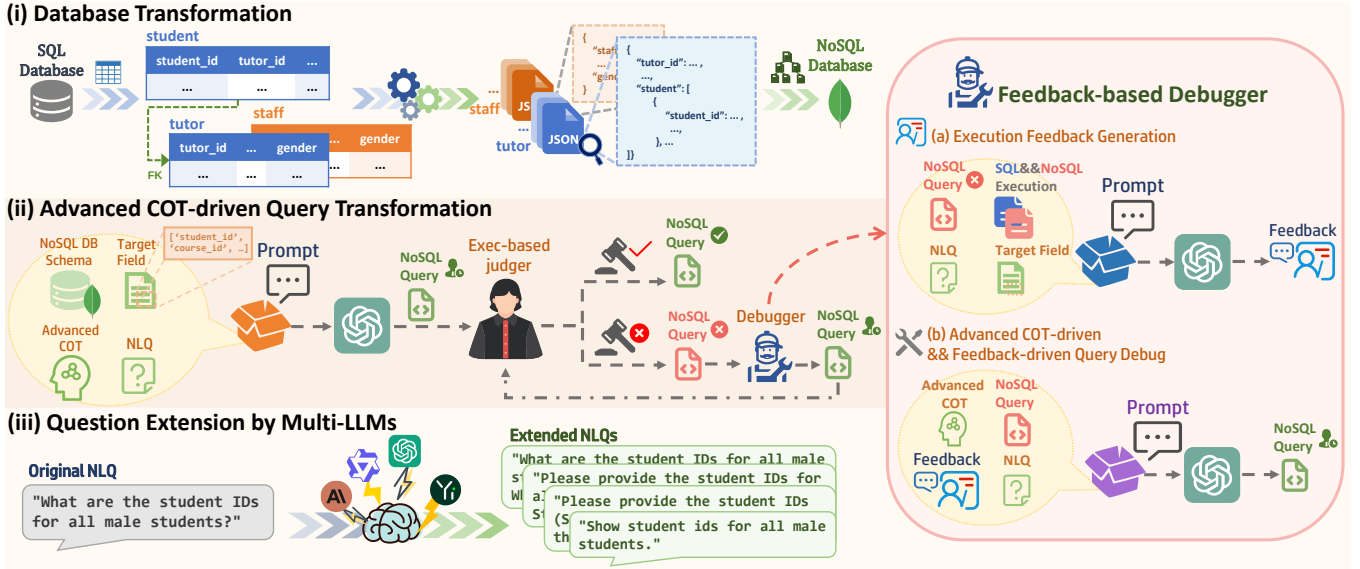


Figure 2: Construction pipeline for the TEND dataset. Through an automated process, the conversion from a Text-to-SQL dataset to a Text-to-NoSQL dataset can be achieved. Specifically, (i) the conversion from SQL databases to NoSQL (e.g., MongoDB) databases is accomplished through algorithmically programmed software; (ii) by inputting examples into the most advanced LLM (such as GPT-4) to obtain examples of Chain-of-Thought, namely Advanced COT, which assists a second-tier LLM (such as GPT-3.5) in reasoning, thus achieving automated generation of NoSQL queries, feedback generation, and query debug; (iii) the expansion of dataset questions is implemented using Multi-LLM.

collection. For instance, in Figure 2(i), the “student” and “tutor” tables are connected by the foreign key “tutor_id”, indicating that these two tables belong to the same relation set, whereas the “staff” table does not have a foreign key connection with other tables, meaning the “staff” table forms its own relation set. For tables within the same relation set (e.g., “student” and “tutor” tables), we identify the table at the far end of the foreign key relationship (e.g., the “tutor” table) and use its column names (e.g., “tutor_id” and “gender” from the “tutor” table) as the outermost keys of the collection. We then begin to traverse all the records in this table, with each record corresponding to a document in the NoSQL database. For each record, we add a key-value pair to store data from other tables linked by foreign keys pointing to this record (e.g., adding the key “student” to store all records from the “student” table where “tutor_id” equals the current “tutor” table record). This process is recursively continued until no more foreign keys point to the currently recursive table. For example, the “student” table is linked to the “tutor” table through the foreign key “tutor_id”. After transformation to a non-relational database, the outermost key-value pairs of the collection will be the records from the “tutor” table, while the records of the “student” table will be stored in the second layer of the collection after being filtered by “tutor_id” in multi-table join queries. Tables connected to the “student” table through foreign keys will also undergo this recursive process, with their records stored in the third layer of key-value pairs in the collection. It is noteworthy that if two tables mutually point to each other through foreign keys, such a database will be discarded

because this situation can lead to an infinite loop in the database transformation process.

2.3 Advanced CoT-driven Query Transformation

After completing the database conversion, we need to generate the corresponding NoSQL queries based on the NoSQL database. Manually writing all the NoSQL queries for a large-scale dataset is also a labor-intensive and time-consuming task. To automate this process, we designed a specialized LLM-based pipeline that leverages the execution results of SQL queries (rather than the queries themselves) to autonomously generate NoSQL queries. These queries are then executed to obtain feedback, which is subsequently used to refine and debug the queries. Below, we provide a detailed introduction to this Query Transformation pipeline (shown in Figure 2(ii)).

2.3.1 Advanced CoT-driven Query Generation. Using state-of-the-art LLMs (like GPT-4o) for inference is very expensive. To fully leverage the performance of the second-tier LLMs (such as GPT-4o-mini), we adopted an Advanced CoT-driven approach for query generation. Specifically, we provided the LLM with input and output examples as the first round of dialogue, where the input includes an introduction for query generation, NoSQL database schemas, and user questions (the prompt is written in markdown format), and the output contains a chain of thoughts generated by the most advanced LLM (GPT-4o), inspired by the following prompt: “A: Let’s think step by step!”. Notably, during query generation, we also provide the schemas obtained from executing the corresponding SQL queries

of the user questions as a reference, effectively assisting the LLM in pre-completing schema-linking.

2.3.2 Execution Feedback Generation. After the Advanced CoT-driven Query Generation, we obtain many unverified NoSQL Queries, and manually checking all queries for correctness would require a considerable amount of time. To screen all queries automatically, we employ the LLM as an inspector. Specifically, we execute the NoSQL queries and their reference SQL queries obtained from Section 2.3.1 on MongoDB and SQLite databases respectively, and organize the results into JSON format for direct comparison. If the two results are identical, then the NoSQL query is considered correct; if they differ, then LLM feedback is required. For LLM feedback, the execution results of the NoSQL and reference SQL queries (in JSON format), user questions, and the schemas required are combined into prompts, instructing the LLM to perform the following two steps: (i) Examine the differences between the data obtained from executing a NoSQL query and the data obtained from executing the corresponding reference SQL query. (ii) Analyze where these differences may have originated from, with no solutions. It is worth noting that the Feedback Generation step also uses the Advanced CoT-driven method, similar to that mentioned in Section 2.3.1.

2.3.3 Advanced CoT-driven and Feedback-driven Query Debug. For those NoSQL queries whose execution results are inconsistent with the reference SQL queries, we need to correct them. To automate NoSQL debugging, we let the LLM act as an inspector in Section 2.3.2, identifying potential issues with the problematic NoSQL queries. We then composed these factors that could cause discrepancies between the execution results of the NoSQL queries and the reference SQL queries into debug prompts, and used the most advanced LLM to generate a debug thought chain as a demonstration dialogue. Subsequently, we let a second-tier LLM correct these errors while continuously checking if the debugged NoSQL query’s execution results are correct. If correct, we exit the debug program and output the NoSQL query; if incorrect, we re-try. After two accumulated errors, we opt to use the highest-level LLM for debugging. If the highest-level LLM still fails to correct the NoSQL query, we abandon that case. It is noteworthy that the execution results of SQL queries serve as a critical dependency in this process, which makes it inherently unsuitable for direct application in Text-to-NoSQL tasks.

2.4 Question Extension by Multi-LLM

Figure 2(iii) illustrates the process of expanding NLQs based on Multi-LLM. To construct a dataset of sufficient size for model training, we need to expand the original NLQs to create enough (NLQ, NoSQL) pairs. In this process, we use LLMs to automatically expand the NLQs from the original Text-to-SQL dataset. We provide the LLMs with NoSQL database schemas, NoSQL queries, reference question examples, and the target schemas for the questions. A single LLM often generates the same questions repeatedly, failing to effectively expand the question set. To address this issue, we employed a Multi-LLM approach, using multiple LLMs (e.g., gpt-3.5-turbo-16k-0613, gpt-4o-2024-05-13 and claude-3-sonnet-20240229) to expand the questions, ultimately resulting in a large-scale Text-to-NoSQL dataset.

#-Databases	#-Collections	#-Domains
154	347	105

Top-5 Domains				
Sport	Customer	School	Shop	Student

The number of Fields and Documents			
#-Fields	#-Avg(#-Fields)	#-Max(#-Fields)	#-Min(#-Fields)
5960	38.70	331	7
#-Docs	#-Avg(#-Docs)	#-Max(#-Docs)	#-Min(#-Docs)
32979	214.15	13694	3

(a) Database Statistics

Query Method		
#-Total	#-Find	#-Aggregate
17020	2770	14250

Operations in Find Method			
#-Filter	#-Projection	#-Sort	#-Limit
2690	2005	600	10

Stages in Aggregate Method				
#-Project	#-Unwind	#-Group	#-Match	#-Sort
13235	8575	7740	7305	3325
#-Limit	#-Lookup	#-Count	#-Others	
2390	2155	955	25	

(b) (NLQ, Query) pairs Statistics

Table 1: TEND Dataset Statistics

2.5 Manual Review and Refinement

Finally, we conducted a comprehensive manual review and refinement of the Text-to-NoSQL dataset TEND generated by the automated pipeline. Specifically, we carefully modified the renamed fields in the NoSQL queries within TEND. For example, in the group stage, which uses aggregation operations similar to those in SQL, such as “sum_Population: { \$sum: "\$Population" }”, the LLM might use different names for “sum_Population”, such as “total_population”. If the renamed fields in the NoSQL queries are not standardized, it could lead to poor performance of the models trained on this dataset in query-based metrics (Section 5.1.3). Therefore, we standardized the naming of aggregation operations, renaming fields to “[operation]_object” (e.g., “sum_population”). Similar to the renaming in the group stage, we also standardized the renaming of the new document collections after joins in the lookup stage, using names like “Docs1” and “Docs2”.

3 THE TEND DATASET: SETUP AND STATISTICS

3.1 Setup

We applied the dataset construction process introduced in Section 2 to the training set of Spider [46], a popular and complex large-scale benchmark dataset for the Text-to-SQL task. The training set includes 166 databases (with an average of 5.28 tables per database) and 7,000 (NLQ, SQL) pairs on these databases. These (NLQ, SQL) pairs are designed to comprehensively cover different databases and various levels of difficulty in Text-to-SQL tasks.

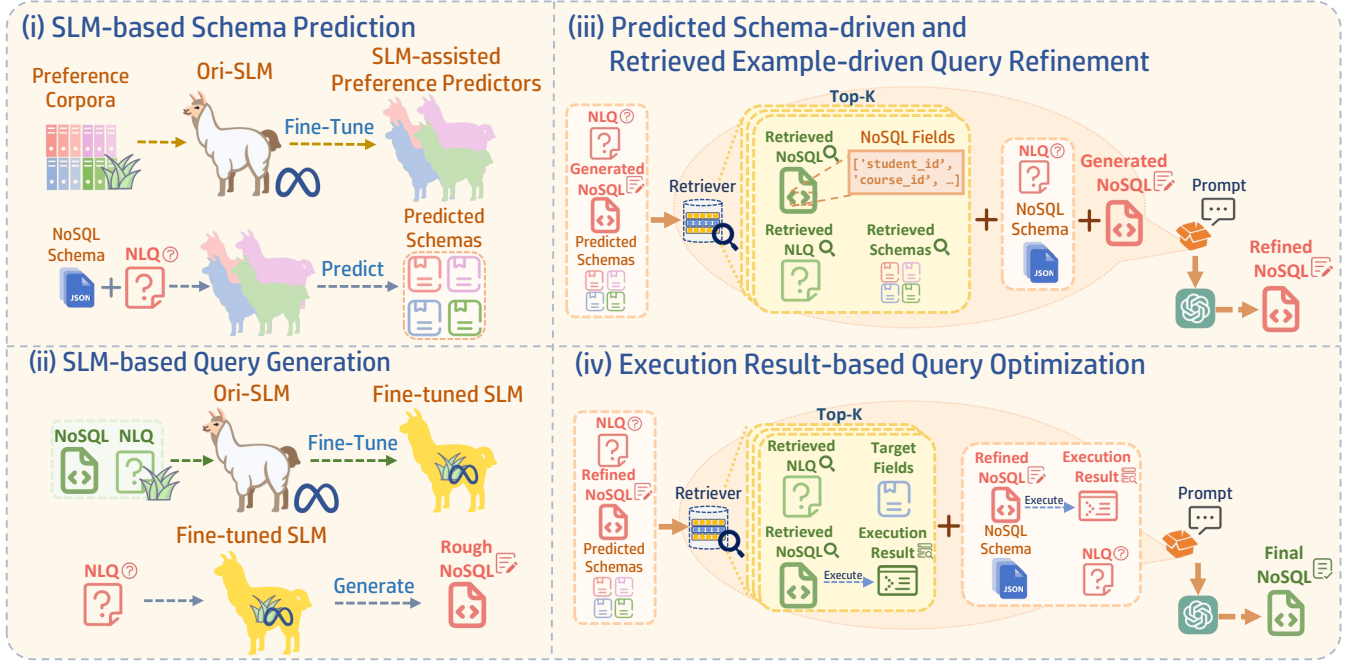


Figure 3: The working pipeline of our proposed SMART framework, where (i) *SLM-based Schema Prediction* predicts the required NoSQL schemas by fine-tuning the SLM. (ii) *SLM-based Query Generation* generates initial NoSQL queries by fine-tuning the SLM. (iii) *Predicted Schema-driven and Retrieved Example-driven Query Refinement* is responsible for refining the NoSQL queries generated by the SLM. The RAG technique used here is detailed in Section 4.3. (iv) *Execution Result-based Query Optimization* optimizes the refined NoSQL queries based on their execution results.

After applying the construction process introduced in Section 2 to Spider, we obtained a high-quality Text-to-NoSQL dataset. However, during manual inspection, we found that some NoSQL queries contained special operations such as “\$isArray”, “\$concatArrays”, and “\$arrayElemAt”. These special operations often introduce unnecessary difficulties for model prediction, similar to special operations like “strftime()” in Text-to-SQL tasks, which should be avoided. To address this issue, we developed a NoSQL query parser using Python, which is responsible for parsing operations in NoSQL queries into JSON data. For example, it can parse the query stages in the NoSQL query “db.people.aggregate([{\$group: { _id: “\$Country”, count: {\$sum: 1} }}, {\$project: { Country: “\$ _id”, count: 1, _id: 0 } }]);” into a list of dictionaries, where each dictionary represents a stage in the query, such as [{\$group: { . . . }}, {\$project: { . . . }}]. To simplify the implementation of this process, we used the demjson library in Python to loosely convert strings into JSON data without requiring the strings to strictly follow JSON format. For instance, “{\$project { _id: 0 }}” cannot be directly decoded into JSON because it lacks necessary quotation marks, but demjson can automatically complete it and parse it into JSON data “{\$project: { “ _id ”: 0 }}”. Using this parser, we can write Python programs to analyze the parsed JSON data of NoSQL queries and filter them based on their operation keywords, directly discarding examples that contain special operations, thereby obtaining the final TEND.

3.2 Statistics of TEND

Table 1 presents the statistical overview of the TEND dataset, encompassing both database and NoSQL query statistics (each NoSQL query corresponds to 5 NLQs, forming 5 pairs of (NLQ, NoSQL)). As illustrated in Table 1(a), the TEND dataset includes 154 databases, which collectively store 347 collections and span 105 domains. The top five domains are sport, customer, school, shop, and student. In total, the databases contain 5,960 fields (corresponding to 32,979 documents), with an average of 38.70 fields (214.15 documents) per database. The database with the highest number of fields (documents) contains 331 fields (13,694 documents), whereas the one with the fewest has only 7 fields (3 documents).

Table 1(b) presents the query statistics for the TEND dataset, which includes a total of 17,020 (NLQ, NoSQL) pairs. Within this dataset, 2,770 pairs utilize the find method, while 14,250 pairs employ the aggregate method. Among the find method pairs, specific operations are distributed as follows: 2,690 pairs incorporate the filter operation, 2,005 utilize the projection operation, 600 employ the sort operation, and 10 pairs apply the limit operation.

For the aggregate method pairs, operations are executed through pipelines, where each operation stage is defined within the pipeline. The data reveal that 13,235 of these aggregate pipelines incorporate the Project stage. Additional operations are utilized across the following stages: Unwind (8,575 pairs), Group (7,740 pairs), Match (7,305 pairs), Sort (3,325 pairs), Limit (2,390 pairs), Lookup (2,155 pairs), Count (955 pairs), and other operations (25 pairs).

Algorithm 2: The SMART Algorithm

Inputs : NLQ list in Test Set Q ;
Database list in Test Set $\mathcal{D}$;
NLQ list in Training Set Q' ;
Database list in Training Set $\mathcal{D}'$;
NoSQL list in Training Set $\mathcal{N}'$;

Output: NoSQL list $\mathcal{N}$

```

1 Procedure SMART( $Q, \mathcal{D}$ ):
  // SLM Fine-tuning
2   $\mathcal{M}_{\text{schema}}, \mathcal{M}_{\text{nosql}} \leftarrow \text{TrainSLM}(\text{SLM}, Q', \mathcal{D}', \mathcal{N}')$ 
  // Build Vector Library
3   $\mathcal{V} \leftarrow \text{BuildVecLib}(Q', \mathcal{D}', \mathcal{N}')$ 
  // Pipeline of SMART
4   $\mathcal{N} \leftarrow []$ 
5  for each  $(q, d) \in (Q, \mathcal{D})$  do
    // SLM-based Schemas Prediction
6     $\mathcal{S} \leftarrow \text{SLMPredict}(\mathcal{M}_{\text{schema}}, q, d)$ 
    // SLM-based Query Generation
7     $n_{\text{gen}} \leftarrow \text{SLMPredict}(\mathcal{M}_{\text{nosql}}, q, d)$ 
    // Query Adjustment
8     $\mathcal{E}_{\text{adj}} \leftarrow \text{Retrieve}(q, \mathcal{S}, n_{\text{gen}}, \mathcal{V})$ 
9     $n_{\text{adj}} \leftarrow \text{QueryAdjust}(q, d, \mathcal{S}, \mathcal{E}_{\text{adj}})$ 
    // Query Optimization
10    $e \leftarrow \text{Execute}(n_{\text{adj}}, d)$ 
11    $\mathcal{E}_{\text{opt}} \leftarrow \text{Retrieve}(q, \mathcal{S}, n_{\text{adj}}, \mathcal{V})$ 
12    $n_{\text{opt}} \leftarrow \text{QueryOptimize}(n_{\text{adj}}, d, e, \mathcal{E}_{\text{opt}})$ 
13    $\mathcal{N}.\text{append}(n_{\text{opt}})$ 
14 end
15 return  $\mathcal{N}$ 

```

4 MULTI-STEP FRAMEWORK FOR TEXT-TO-NOSQL: SLM AND RAG ASSISTANCE

To provide a high-performance solution for the novel task of Text-to-NoSQL, we have designed a Multi-step framework called SMART (an abbreviation for SLM-assisted and RAG-assisted Multi-step framework) based on RAG techniques and fine-tuned SLM. This chapter will provide a detailed introduction to SMART.

Overview: As shown in Figure 3 and Algorithm 2, SMART is mainly divided into four processes: SLM-based Schema Prediction (Section 4.1), SLM-based Query Generation (Section 4.2), Predicted Schema-driven and Retrieved Example-driven Query Refinement (Section 4.3), and Execution Result-based Query Optimization (Section 4.4). Compared to LLM, the fine-tuning cost of SLM is much lower, making SLM a promising auxiliary tool. Especially when we need to predict code or details that are easily influenced by preferences (e.g. the renamed fields in NoSQL queries), LLM often struggles to align with these preferences. This means that directly using prompting methods makes it difficult for LLM to achieve high accuracy. However, with the assistance of SLM, the entire framework can achieve this alignment. RAG technology is a mainstream auxiliary technology for LLM, but its drawback lies in its high requirements for retrieval technology. It requires retrieving high-quality examples and adding them to the prompt context to be effective. If the quality of the context examples is poor, it can

significantly interfere with LLM’s predictions. To maximize the relevance of the retrieved examples, the retrieval method used in the SMART framework does not solely rely on the similarity of NLQs between examples. Instead, it calculates the final example similarity by weighting and summing the cosine similarities of NLQs, NoSQL queries, and schemas (such as fields in the queries and fields displayed in the execution result documents) to retrieve examples. Below, we introduce the four main processes of SMART in detail.

4.1 SLM-based Schema Prediction

As shown in Figure 3(i), the implementation of schema prediction based on SLM is accomplished through model fine-tuning. Specifically, we parse each example in the training set of TEND, extracting various schema information from the NoSQL queries, such as the collections queried, the database fields queried, the renamed fields in the query, and the fields included in the resulting documents after query execution. These schema details are then paired with the corresponding NLQ to form prompts and outputs, thereby constructing a fine-tuning corpus for predicting various schemas. We then use this fine-tuning corpus to fine-tune SLMs specialized in predicting different schemas. Finally, we use these SLMs to perform schema prediction on examples in the TEND test set.

```

1 You are now the MongoDB natural language interface, responsible
  for converting user input natural language queries into
  MongoDB query statements based on the MongoDB Collection
  and their Fields, and parsing the features according to
  user requirements.
2 Human: # Given the natural language query, please predict the
  fields used in the query.
3 ## Natural Language Query: "<NLQ>"
4 ## MongoDB Collection and their Fields
5 <MongoDB_Schemas>
6 Assistant:

```

The specific prompt template in the corpus is as shown above. It consists of four main parts: the system prompt, the instruction, the NLQ, and the complete database schemas. The output is the serialized schema information. For example, in the NoSQL query “db.train.find({}, {"Name":1, "Time":1, "Service":1, "_id":0});”, the database fields used are “Name, Service, Time, _id”.

4.2 SLM-based Query Generation

Figure 3(ii) illustrates the SLM-based Query Generation process. This process involves directly constructing the TEND training set into a text-to-NoSQL corpus and using this corpus to fine-tune a specialized text-to-NoSQL SLM. Subsequently, this SLM is used to directly predict examples in the test set, generating initial NoSQL queries.

The following is the prompt template used for fine-tuning the SLM in the SLM-based Query Generation process. Similar to the template for SLM-based Query Generation, it consists of the system prompt, the instruction, the NLQ, and the database schemas. A specific example is as follows: the SLM needs to convert the NLQ “Count the number of products.” into the NoSQL query “db.Ref_Colors.aggregate([{\$unwind:"\$Products"},{\$group: {_id:null, count:{\$sum:1}}},{\$project: {_id:0, count:1}}]);”

```

1 You are now the MongoDB natural language interface, responsible
  for converting user input natural language queries into
  MongoDB query statements based on the MongoDB collections
  and their fields.
2 Human: # Given the MongoDB collections and their fields and
  natural language query, please generate final MongoDB query.

3 ## Natural Language Query: `Count the number of schools that have
  had basketball matches.`
4 ## MongoDB Collection and their Fields
5 <MongoDB_Schemas>
6 Assistant:

```

4.3 Predicted Schema-driven and Retrieved Example-driven Query Refinement

Figure 3(iii) illustrates the process of query refinement based on predicted schemas and queries. Specifically, we first compute the cosine similarity of each element (natural language query (NLQ), schema predicted by the SLM-based schema predictor, and initial NoSQL query generated by the SLM-based query generator) with examples in the training set. We then perform a weighted sum to retrieve the top k most relevant reference examples. The formula for calculating sample similarity is as follows:

$$\text{Sim} = \text{Sim}_{\text{nlq}} \times w_1 + \text{Sim}_{\text{nosql}} \times w_2 + \dots$$

where Sim represents the cosine similarity, w represents the weights, with “ $w_1=1.0$, $w_2=0.3$ ”. The reason for this weighting is that the NLQ is an actual value, whereas the NoSQL query and other elements are predicted values. It is worth noting that the sum of the weights for all similarity measures does not necessarily have to be 1. This is because the final ranking of similarities is based on the magnitude of Sim . When all Sim values are proportionally scaled by a factor of w , the results remain unaffected. For example, a ratio of $1.0 : 0.5 : 0.5$ (where the sum of weights is not equal to 1) is equivalent to $0.5 : 0.25 : 0.25$ (where the sum of weights equals 1). The “...” represents the weighted sum of the similarities of database fields and the fields displayed in the document, which are also multiplied by the weight w_2 but are omitted due to space constraints. The retrieved reference examples are subsequently used to form the context for the RAG prompt. The LLM first evaluates whether the NoSQL query generated by the SLM is reasonable given the current NLQ and database schema. If it is not reasonable, the LLM adjusts the query based on the retrieved examples; if it is reasonable, the original NoSQL query is retained. The reason for adopting this retrieval strategy is to maximize the relevance of the retrieved examples to the current example, thereby enhancing the LLM’s ability of refining the NoSQL query.

```

1 ## Query Transformation Reference Examples
2 [RAG_examples]
3 ## MongoDB collections and their fields
4 [MongoDB_Schemas]
5 ## Natural Language Query
6 - `[NLQ]`
7 ## Original MongoDB Query
8 [NoSQL]
9 [Predicted_Schemas]
10 [instruction]
11 A: Let's think step by step!

```

The above is the prompt template for query refinement based on predicted schemas and queries.

4.4 Execution Result-based Query Optimization

After the query adjustment in Section 4.3, we also designed a module for query optimization based on execution results (Figure 3(iv)). This module primarily relies on RAG technology. Consistent with the retrieval strategy introduced in Section 4.3, the query optimizer also relies on the cosine similarity of elements such as NLQ and schemas to retrieve examples. For each retrieved example, we execute the NoSQL query to obtain the execution result document, allowing the LLM to reference the mapping relationships between the NLQ, NoSQL query, and execution results to further optimize the NoSQL query. The following is the prompt template for this step:

```

1 ## Reference Examples:
2 [RAG_Examples]
3 ## MongoDB collections and their fields
4 [MongoDB_Schemas]
5 ## Natural Language Query
6 - `[NLQ]`
7 ## Fields Shown in the Execution Results
8 - `[target_fields]`
9 ## Original MongoDB Query
10 [NoSQL]
11 ### Execution Results
12 [Exec_Results]
13 [instruction]
14 A: Let's think step by step!

```

5 EXPERIMENTS AND ANALYSIS

This section shows extensive experimental results on a new dataset (TEND). We provide a detailed description of the experimental setup, evaluation metrics, and baseline models, followed by a comparative analysis of the performance between our proposed model (SMART) and the baseline models. Subsequently, we perform ablation studies and parameter experiments to validate the effectiveness of each component in SMART and explore the optimal hyperparameter settings. Finally, we present detailed case studies through specific examples to illustrate the concrete performance of each method.

5.1 Experimental Setup

5.1.1 Dataset. We have partitioned the TEND dataset into training and testing sets according to the cross-domain standard, with a ratio of approximately 8:2. The resulting training and testing sets contain 14,245 and 2,775 pairs of (NLQ, NoSQL) instance, respectively. This division of the dataset allows for an analysis of how each method performs when confronted with unseen data, thereby providing a more comprehensive evaluation of the model’s performance.

5.1.2 Models. We utilized a variety of popular neural network models and LLM-based prompting methods as baseline models for a comprehensive performance comparison with SMART. The models are as follows:

- **Seq2Seq:** The Seq2Seq [1] model transforms natural language queries and database schemas into hidden states via an encoder, and then generates NoSQL queries through a decoder.
- **Transformer:** Transformer [40] demonstrated excellent performance in tasks such as machine translation [5], dialogue systems [48], and speech recognition [47], and it is also a mainstream model in neural network models.

- **Instructing LLM:** The Instructing LLM is an experimental setup designed to guide the LLM in performing Text-to-NoSQL predictions by crafting clear and precise instructional prompts.
- **Few-shot LLM:** The few-shot prompting method is an important way to implement in-context learning (ICL), by enhancing a certain number of examples in the context to teach LLMs how to perform tasks in specific domains.
- **RAG for LLM:** RAG technology can adaptively select examples from the knowledge base as references based on the input, which can further mitigate model hallucinations.
- **Fine-tuned Llama:** The Fine-tuned Llama is an experimental setup that involves supervised fine-tuning of Llama-3.2-1B using the training set of TEND, representing the performance of model training-based approaches on the Text-to-NoSQL task.
- **SMART:** SMART is the framework proposed in this paper, which, with the assistance of SLM and RAG technologies, constructs four main processes: SLM-based Query Generation, SLM-based Query Generation, query refinement based on predicted schema and retrieved examples, and execution results-based query optimization. To validate the versatility of SMART, we conducted experiments on both gpt-4o-mini and deepseek-v3.

5.1.3 Evaluation Metrics. To evaluate the performance of models on the Text-to-NoSQL task, we introduce several metrics:

- **Exact Match (EM):** This metric evaluates whether the generated query exactly matches the gold query in both structure and content. It is calculated as:

$$EM = N_{em} / N$$

where N_{em} is the number of queries that fully match the gold query, and N is the total number of queries in the test set. EM provides a strict measure of syntactic and semantic alignment.

- **Query Stages Match (QSM):** QSM assesses whether the key stages (e.g., match, group, lookup) in the generated query match the gold query in terms of order and keywords. It is computed as:

$$QSM = N_{qsm} / N$$

where N_{qsm} is the number of queries with matching stages.

- **Query Fields Coverage (QFC):** QFC measures whether the fields in the generated query cover all fields in the gold query, including database fields and query-defined fields. It is defined as:

$$QFC = N_{qfc} / N$$

where N_{qfc} is the number of queries with complete field coverage.

- **Execution Accuracy (EX):** This metric evaluates the correctness of the results obtained by executing the generated query on the database. It is calculated as:

$$EX = N_{ex} / N$$

where N_{ex} is the number of queries whose execution results match those of the gold query. EX is the most critical performance metric for evaluating Text-to-NoSQL models.

- **Execution Fields Match (EFM):** EFM checks whether the field names in the execution results of the generated query match those of the gold query. It is computed as:

$$EFM = N_{efm} / N$$

where N_{efm} is the number of queries with matching field names in the results.

- **Execution Value Match (EVM):** EVM measures whether the values in the execution results of the generated query match those of the gold query. It is defined as:

	Query-based Metric		
Method	EM	QSM	QFC
Seq2Seq	0.00%	0.00%	0.00%
Transformer	0.00%	0.00%	0.00%
Instructing LLM	5.91%	50.77%	64.32%
Few-shot LLM	10.41%	48.40%	55.46%
RAG for LLM	16.32%	63.06%	70.05%
Fine-tuned Llama	20.54%	56.50%	67.68%
SMART (gpt-4o-mini)	21.77%	60.65%	72.86%
SMART (deepseek-v3)	23.82%	63.21%	75.60%

(a) Query-based Metric Results

	Execution-based Metric		
Method	EX	EFM	EVM
Seq2Seq	0.00%	0.00%	0.00%
Transformer	0.00%	0.00%	0.00%
Instructing LLM	35.06%	53.05%	58.23%
Few-shot LLM	35.82%	66.99%	68.25%
RAG for LLM	53.26%	73.95%	62.02%
Fine-tuned Llama	53.12%	84.36%	68.11%
SMART (gpt-4o-mini)	59.57%	84.90%	69.73%
SMART (deepseek-v3)	65.08%	87.21%	72.79%

(b) Execution-based Metric Results

Table 2: Performance Comparison

$$EVM = N_{evm} / N$$

where N_{evm} is the number of queries with matching values in the results.

5.1.4 Implementation Details. The SLM used in SMART and Fine-tuned Llama is Llama-3.2-1B, which is fine-tuned using a full-parameter fine-tuning strategy with a batch size set to 4. The LLM used in Instructing LLM, Few-shot LLM, RAG for LLM, and SMART is deepseek-v3, with the parameter configured as “temperature=0.0”. The text-to-embedding model employed is “text-embedding-ada-002”. The number of examples provided by Few-shot LLM, RAG for LLM, and SMART is 20.

5.2 Performance Comparison

As shown in Table 2, the Seq2Seq and Transformer models were completely unable to achieve Text-to-NoSQL predictions. By analyzing the prediction results of these two models, we found that the model outputs failed to grasp the code logic of special symbols such as ‘()’, ‘[]’, ‘.’, and ‘\$’. This led to a large amount of special symbol redundancy in the model outputs, such as “db ... aggregate ([{ \$: { \$: {”. This issue is caused by the NoSQL syntax structure and represents a significant challenge in the Text-to-NoSQL task.

In addition, the three baseline methods based on the prompting method-Instructing LLM, Few-shot LLM, and RAG for LLM-achieved notable performance in the most critical metric, EX, with RAG for LLM particularly reaching an execution accuracy of 52.76%. The Fine-tuned Llama, obtained through supervised fine-tuning,

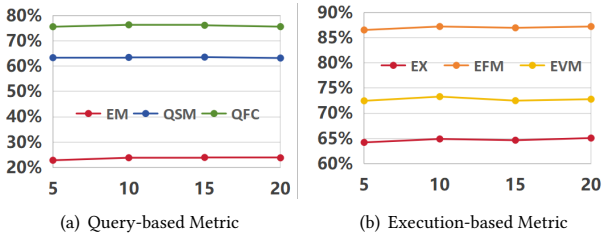


Figure 4: Parameter study. These are the variation curves of SMART under different numbers of retrieval examples and different metrics. The vertical axis represents the model’s accuracy under a specific metric, and the horizontal axis represents the number of retrieval examples.

delivered comparable performance to RAG for LLM, achieving an execution accuracy of 53.12%. These results indicate that even simple methods can yield satisfactory outcomes in the Text-to-NoSQL task, validating the feasibility of the task. The framework we designed, SMART, excelled in this task, achieving an accuracy of 65.08% in Execution Accuracy. This fully demonstrates the advanced nature of SMART. Moreover, SMART achieved the best performance across all detailed metrics, indicating that it outperforms all baseline methods in terms of field usage and operation stages, understanding of field mapping, and data processing.

Text-to-SQL is a well-developed field, and we need to explore the cascading approach of Text-to-SQL and SQL-to-NoSQL to validate the value of the Text-to-NoSQL task. However, due to limited space in the main text, we have placed the discussion and analysis of SQL-to-NoSQL technology in the technical report mentioned in the footnote of Section 1.

5.3 Parameter Study

To explore the performance of SMART under different parameters, we conducted parameter experiments on the test set of the TEND dataset, varying the number of retrieval examples. As shown in Figure 4, we found that as the number of retrieval examples increased, SMART exhibited different performance curves across various metrics. Specifically, the accuracy of SMART on query-based metrics initially decreased and then increased. In contrast, the execution-based metrics showed a fluctuating trend. Overall, EX is the metric that best reflects the model’s performance in real-world scenarios. When focusing on the EX metric, SMART achieved the highest execution accuracy of 65.08% with 20 retrieval examples.

5.4 Ablation Study

In this section, we conduct ablation experiments to examine the effectiveness and contribution of each major process and component in SMART. Specifically, we first evaluate SMART with all major processes included. Then, we remove or replace some key processes in SMART and assess its performance under the following configurations: (i) removing SLM-based Query Generation (**w/o SP**); (ii) removing Predicted Schema-driven and Retrieved Example-driven Query Refinement (**w/o RF**); (iii) removing execution results-based query optimization (**w/o OPT**); (iv) using only SLM-based Query Generation (**only GEN**). Additionally, we include the experimental results of Instructing LLM and RAG for LLM from the table as

	Query-based Metric		
Method	EM	QSM	QFC
Instructing LLM	5.91%	50.77%	64.32%
RAG for LLM	37.74%	62.10%	51.57%
SMART (Ours)	23.82%	63.21%	75.60%
- w/o SP	23.78%	63.53%	76.65%
- w/o RF	24.07%	63.64%	75.28%
- w/o OPT	23.06%	63.21%	75.71%
- only GEN	20.54%	56.50%	67.68%

(a) Query-based Metric Results

	Execution-based Metric		
Method	EX	EFM	EVM
Instructing LLM	35.06%	53.05%	58.23%
RAG for LLM	57.02%	74.61%	71.29%
SMART (Ours)	65.08%	87.21%	72.79%
- w/o SP	63.78%	85.80%	71.24%
- w/o RF	62.85%	87.17%	70.88%
- w/o OPT	63.78%	86.85%	72.47%
- only GEN	53.12%	84.36%	68.11%

(b) Execution-based Metric Results

Table 3: Ablation study. This table shows the performance of SMART on various metrics after removing each component.

part of the ablation study. The purpose is to demonstrate that the high accuracy performance of SMART primarily stems from our designed complex framework rather than solely from the inherent strong performance of the LLM itself.

The results of the ablation experiments are shown in Table 3. The SMART configuration with all processes included performs slightly lower on all metrics except for the Query-based ones compared to other configurations, but it leads in the Execution-based metrics across all experimental setups. The w/o RF configuration performs best on the EM and QSM metrics, indicating that providing the LLM with the query and its execution results during the execution results-based query optimization process allows the LLM to better understand the structure of the query itself. However, on the EX metric, which best reflects the model’s performance in real-world scenarios, SMART with all processes included outperforms all other configurations. This validates that all components in SMART effectively contribute to the overall framework. Furthermore, SMART with all major processes included significantly outperforms the Instructing LLM, RAG for LLM and only GEN configurations on the important EX and EM metrics. This indicates that the high accuracy of SMART primarily stems from the framework itself rather than the understanding and execution capabilities of the LLM or fine-tuned SLM in the Text-to-NoSQL task.

5.5 Case Study

Table 4 presents a case study comparing the NoSQL queries generated by various baseline methods and SMART, along with their execution results. Due to space limitations, we only showcase the performance of the most representative baseline models, including RAG for LLM, which represents prompting methods, and Fine-tuned

NLQ	What are the dates of completion for tests that have received a failing result
Target NoSQL	<pre>db.Subjects.aggregate([{ \$unwind: "\$Courses" }, { \$unwind: "\$Courses.Student_Course_Enrolment" }, { \$unwind: "\$Courses.Student_Course_Enrolment.Student_Tests_Taken" }, { \$match: { "Courses.Student_Course_Enrolment.Student_Tests_Taken.test_result": "Fail" } }, { \$project: { date_of_completion: "\$Courses.Student_Course_Enrolment.date_of_completion", _id: 0 } }]]);</pre> Standard query: Correct usage of nested field paths, returns 7 results Result: [{'date_of_completion': '2018-03-04 01:24:56'}, ...]
RAG for LLM	<pre>db.Subjects.aggregate([{ \$unwind: "\$Courses" }, { \$unwind: "\$Courses.Student_Course_Enrolment" }, { \$unwind: "\$Courses.Student_Course_Enrolment.Student_Tests_Taken" }, { \$match: { "Courses.Student_Course_Enrolment.Student_Tests_Taken.test_result": "Fail" } }, { \$project: { _id: 0, date_test_taken: "\$Courses.Student_Course_Enrolment.Student_Tests_Taken.date_test_taken" } }]]);</pre> Error: Incorrect field name (date_test_taken) used, leading to unexpected results Result: [{'date_test_taken': '2018-03-22 13:10:06'}, ...]
Fine-tuned Llama	<pre>db.Courses.aggregate([{ \$unwind: "\$Courses" }, { \$unwind: "\$Courses.Student_Course_Enrolment" }, { \$unwind: "\$Courses.Student_Course_Enrolment.Student_Tests_Taken" }, { \$match: { "Courses.Student_Course_Enrolment.Student_Tests_Taken.test_result": "Fail" } }, { \$project: { date_of_completion: "\$Courses.Student_Course_Enrolment.date_of_completion", _id: 0 } }]]);</pre> Error: Wrong collection name (Courses instead of Subjects) used, resulting in empty output Result: []
SMART (Ours)	<pre>db.Subjects.aggregate([{ \$unwind: "\$Courses" }, { \$unwind: "\$Courses.Student_Course_Enrolment" }, { \$unwind: "\$Courses.Student_Course_Enrolment.Student_Tests_Taken" }, { \$match: { "Courses.Student_Course_Enrolment.Student_Tests_Taken.test_result": "Fail" } }, { \$project: { date_of_completion: "\$Courses.Student_Course_Enrolment.date_of_completion", _id: 0 } }]]);</pre> Success: Correct query structure and field usage, demonstrating accurate understanding of schema and requirements Result: [{'date_of_completion': '2018-03-04 01:24:56'}, ...]

Table 4: Case Study. Listed are the NoSQL queries generated by SMART alongside two representative baselines—RAG for LLM and Fine-tuned—using the Prompting method and fine-tuning method, respectively, along with their corresponding execution results (errors are marked in red, and correct results are marked in green).

Llama, which represents model training methods. In the example from Table 4, RAG for LLM can correctly generate query stages and the most operations but fails to accurately establish the mapping between the nested structure of the database and the NLQ, leading to the inference of the wrong field “date_test_taken”. On the other hand, Fine-tuned Llama predicted the correct phase and all data processing steps, but it made an error in the simplest part by inferring a collection name that does not exist in the database. Such errors are common in end-to-end models, as they tend to rely heavily on training data. When faced with unfamiliar databases, they are prone to model hallucinations influenced by the training data. In contrast, SMART is able to accurately generate the correct query, demonstrating that SMART can effectively understand and execute the Text-to-NoSQL task and can predict test set examples well after learning from the training set.

6 RELATED WORK

Our work intersects with three key research areas: NoSQL databases, text-to-SQL systems, and natural language interfaces for data systems. In this section, we review relevant studies from each field.

6.1 NoSQL Databases

NoSQL databases have become a crucial component of modern data management systems, primarily due to their flexibility, scalability, and high performance. These attributes are essential for handling the large volumes of unstructured and semi-structured data commonly encountered in contemporary applications. NoSQL databases allow for efficient horizontal scaling and offer robust options for distributed data storage, making them an ideal choice for large-scale applications that require high availability and fault tolerance.

In the fields of databases and data mining, current research focuses on several key areas within NoSQL databases. These include enhancing scalability and reliability in distributed environments [4], improving query capabilities for efficient large-scale data processing [24], and ensuring data security and privacy in the context of increasing cloud deployments [27]. Additionally, many scholars are exploring the differences between NoSQL and SQL, aiming to uncover the development potential and future directions of NoSQL technologies.

Despite the maturation of NoSQL-related technologies, challenges remain in terms of accessibility and usability, particularly

for users unfamiliar with specific query languages. This paper introduces the Text-to-NoSQL task, which aims to enable users to interact with databases using intuitive NLQs. This technology not only broadens the user base but also enhances the efficiency and productivity of data management operations by bridging the gap between technical and user language capabilities.

6.2 Text-to-SQL

Text-to-SQL aims to automatically convert NLQs into SQL queries, effectively bridging the gap between non-expert users and traditional relational database systems. This task has been extensively studied within both the database and natural language processing communities. Early research efforts relied on predefined rules or query enumeration to tackle the Text-to-SQL task [2, 32, 35], or treated it as a sequence-to-sequence problem, leveraging encoder-decoder architectures to predict the final SQL queries [3, 29, 31]. With the rapid advancement of deep learning, numerous techniques such as attention mechanisms [23], graph representations [15, 20, 31, 43, 45, 49], and syntax parsing [13, 18, 34, 44] have been employed to aid the Text-to-SQL task, resulting in significant improvements. To evaluate the performance of Text-to-SQL models in real-world scenarios, several large-scale benchmark datasets have been constructed and released, including WikiSQL [50], Spider [46], KaggleDBQA [17], and BIRD [21].

With the advent of large language models (LLMs), the Text-to-SQL task has entered a new era of development [7, 10, 12, 19, 25, 30, 33]. By pre-training on massive text corpora, LLMs have developed strong natural language understanding capabilities, enabling them to accomplish various downstream tasks with input prompts. The paradigm of leveraging LLMs for Text-to-SQL tasks has emerged as the new mainstream approach. For instance, Ren et al. [33] designed a more intricate pipeline involving Schema Pruning, Skeleton Prediction, and Demonstration Selection to generate SQL queries from LLM prompts. They further refined the SQL output by adjusting it to fit specific database schemas and SQL dialects and addressed hallucination issues.

Although significant progress has been made in Text-to-SQL conversion technology, research in the field of Text-to-NoSQL is still in its infancy. Moreover, due to substantial differences in database structures, query languages, and application scenarios, existing Text-to-SQL research cannot be directly applied to Text-to-NoSQL. Specifically, NoSQL databases typically feature more complex schemas, and query syntax varies significantly across different systems, posing substantial challenges for SQL-to-NoSQL conversion. As a result, achieving Text-to-NoSQL through a cascaded approach of Text-to-SQL and SQL-to-NoSQL is not only technically demanding but also resource-intensive. This conclusion has been experimentally validated in the technical report cited in the footnote of Section 1.

6.3 Natural Language Interfaces for Data Systems

The application of Natural Language Interfaces to Data Systems (NLIDS) in the industry has been evolving for a considerable period. Numerous applications have demonstrated the practicality of natural language interfaces in data interaction. For instance, Salesforce

Einstein integrates natural language processing, enabling users to query Customer Relationship Management (CRM) data using simple language. Amazon QuickSight allows users to pose questions about their datasets without complex query syntax. Google Cloud’s Natural Language API assists businesses in classifying and understanding text data. IBM Watson Assistant provides a conversational interface, allowing users to interact with backend systems via natural language. Microsoft Power BI features a Q&A function that translates NLQs into visual data insights. These applications are designed to enhance data accessibility for non-technical users.

In the academic realm, research on NLIDS is similarly well-explored. Studies such as WikiSQL [51] and Spider [46] have provided large-scale datasets to evaluate models that convert NLQs into SQL queries, assisting the examination of this conversion process in real-world systems. Ni et al. [26] and Staniek et al. [39] have explored the conversion of NLQs into query languages for graph databases and the Overpass geographic database, respectively, further expanding the research boundaries of NLIDS. Furthermore, Song et al.’s work [37, 38] extends NLIDS model inputs from text to audio, highlighting further innovation in the field. A survey on NLIDS reviewed the current state of this technology and the open challenges it faces [28].

However, these studies have not directly addressed research on NLIDS for NoSQL databases. Therefore, in this work, we introduce a large-scale dataset, TEND, aimed at evaluating the ability of models to translate NLQs into query languages specific to NoSQL databases. This initiative is intended to foster further research on NLIDS for NoSQL databases within the database and data mining communities. In addition, we have developed SMART, a novel system, and have conducted extensive experiments to demonstrate the feasibility of this new task.

7 CONCLUSION AND FUTURE WORK

This study introduces the Text-to-NoSQL task for the first time, aiming to generate NoSQL queries from natural language queries to lower the barrier for non-technical users. To this end, we developed a novel dataset construction process and released a large-scale, open-source benchmark dataset called TEND. We also designed a multi-step framework based SLM and RAG, and established a set of evaluation metrics to measure model performance based on both the query and its execution results. Experimental results demonstrate the feasibility of the Text-to-NoSQL task and highlight the significant progress achieved by our proposed SMART model.

Although TEND provides a competitive benchmark platform for Text-to-NoSQL research and propels the advancement in this field, it is worth noting that its databases are converted from relational databases through graph algorithms. Therefore, constructing new Text-to-NoSQL datasets based on NoSQL databases from real-world application scenarios represents a promising and practically significant direction for future research.

In addition, SMART has demonstrated robust performance when dealing with unknown databases, significantly outperforming various end-to-end models such as Fine-tuned Llama. However, efficiency is also a crucial factor in practical applications. Exploring ways to enhance efficiency without compromising accuracy is a highly promising direction.

ACKNOWLEDGE

Yuanfeng Song is the corresponding author.

REFERENCES

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *3rd International Conference on Learning Representations, ICLR 2015*.
- [2] Christopher Baik, Zhongjun Jin, Michael Cafarella, and H. V. Jagadish. 2020. Duoquest: A Dual-Specification System for Expressive SQL Queries. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) (*SIGMOD '20*). Association for Computing Machinery, New York, NY, USA, 2319–2329. <https://doi.org/10.1145/3318464.3389776>
- [3] Ruichu Cai, Boyan Xu, Zhenjie Zhang, Xiaoyan Yang, Zijian Li, and Zhihao Liang. 2018. An encoder-decoder framework translating natural language to database queries. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence* (Stockholm, Sweden) (*IJCAI'18*). AAAI Press, 3977–3983.
- [4] Rick Cattell. 2011. Scalable SQL and NoSQL data stores. *SIGMOD Rec.* 39, 4 (may 2011), 12–27. <https://doi.org/10.1145/1978915.1978919>
- [5] Anna Currey and Kenneth Heafield. 2019. Incorporating source syntax into transformer-based neural machine translation. In *Proceedings of the Fourth Conference on Machine Translation (Volume 1: Research Papers)*, 24–33.
- [6] Ali Davoudian, Liu Chen, and Mengchi Liu. 2018. A survey on NoSQL stores. *ACM Computing Surveys (CSUR)* 51, 2 (2018), 1–43.
- [7] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, lu Chen, Jinshu Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. arXiv:2307.07306 [cs.CL] <https://arxiv.org/abs/2307.07306>
- [8] Catherine Finegan-Dollak, Jonathan K. Kummerfeld, Li Zhang, Karthik Ramnathan, Sesh Sadasivam, Rui Zhang, and Dragomir Radev. 2018. Improving Text-to-SQL Evaluation Methodology. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Iryna Gurevych and Yusuke Miyao (Eds.). Association for Computational Linguistics, Melbourne, Australia, 351–360. <https://doi.org/10.18653/v1/P18-1033>
- [9] Deka Ganesh Chandra. 2015. BASE analysis of NoSQL database. *Future Gener. Comput. Syst.* 52, C (nov 2015), 13–21. <https://doi.org/10.1016/j.future.2015.05.003>
- [10] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (may 2024), 1132–1145. <https://doi.org/10.14778/3641204.3641221>
- [11] Felix Gessert and Norbert Ritter. 2016. Scalable data management: NoSQL data stores in research and practice. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. IEEE, 1420–1423.
- [12] Zihui Gu, Ju Fan, Nan Tang, Lei Cao, Bowen Jia, Sam Madden, and Xiaoyong Du. 2023. Few-shot Text-to-SQL Translation using Structure and Content Prompt Learning. *Proc. ACM Manag. Data* 1, 2, Article 147 (jun 2023), 28 pages. <https://doi.org/10.1145/3589292>
- [13] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Anna Korhonen, David Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, Florence, Italy, 4524–4535. <https://doi.org/10.18653/v1/P19-1444>
- [14] Jing Han, Haihong E, Guan Le, and Jian Du. 2011. Survey on NoSQL database. In *2011 6th International Conference on Pervasive Computing and Applications*. 363–366. <https://doi.org/10.1109/ICPCA.2011.6106531>
- [15] Binyuan Hui, Ruiying Geng, Lihan Wang, Bowen Qin, Yanyang Li, Bowen Li, Jian Sun, and Yongbin Li. 2022. S²SQL: Injecting Syntax to Question-Schema Interaction Graph Encoder for Text-to-SQL Parsers. In *Findings of the Association for Computational Linguistics: ACL 2022*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 1254–1262. <https://doi.org/10.18653/v1/2022.findings-acl.99>
- [16] JavalnUse. 2023. SQL to MongoDB Query Converter. <https://www.javainuse.com/sql2mongo>. Accessed: 2025-1-11.
- [17] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic Evaluation of Text-to-SQL Parsers. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 2261–2273. <https://doi.org/10.18653/v1/2021.acl-long.176>
- [18] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirtieth Symposium on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1466, 9 pages. <https://doi.org/10.1609/aaai.v37i11.26535>
- [19] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. CodeS: Towards Building Open-source Language Models for Text-to-SQL. *Proc. ACM Manag. Data* 2, 3, Article 127 (may 2024), 28 pages. <https://doi.org/10.1145/3654930>
- [20] Jinyang Li, Binyuan Hui, Reynold Cheng, Bowen Qin, Chenhao Ma, Nan Huo, Fei Huang, Wenyu Du, Luo Si, and Yongbin Li. 2023. Graphix-T5: mixing pre-trained transformers with graph-aware layers for text-to-SQL parsing. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirtieth Symposium on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1467, 9 pages. <https://doi.org/10.1609/aaai.v37i11.26536>
- [21] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Ma Chenhao, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A Blg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 42330–42357. https://proceedings.neurips.cc/paper_files/paper/2023/file/83fc8fab1710363050bbdd1d4b8cc0021-Paper-Datasets_and_Benchmarks.pdf
- [22] Yishan Li and Sathiamoorthy Manoharan. 2013. A performance comparison of SQL and NoSQL databases. In *2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*. 15–19. <https://doi.org/10.1109/PACRIM.2013.6625441>
- [23] Hu Liu, Yuliang Shi, Jianlin Zhang, Xinjun Wang, Hui Li, and Fanyu Kong. 2023. Multi-hop Relational Graph Attention Network for Text-to-SQL Parsing. In *2023 International Joint Conference on Neural Networks (IJCNN)*. 1–8. <https://doi.org/10.1109/IJCNN54540.2023.10191914>
- [24] Divya Mahajan, Cody Blakeney, and Ziliang Zong. 2019. Improving the energy efficiency of relational and NoSQL databases via query optimizations. *Sustainable Computing: Informatics and Systems* 22 (2019), 120–133. <https://doi.org/10.1016/j.suscom.2019.01.017>
- [25] Linyong Nan, Yilun Zhao, Weijin Zou, Narutatsu Ri, Jaesung Tae, Ellen Zhang, Arman Cohan, and Dragomir Radev. 2023. Enhancing Text-to-SQL Capabilities of Large Language Models: A Study on Prompt Design Strategies. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 14935–14956. <https://doi.org/10.18653/v1/2023.findings-emnlp.996>
- [26] Pin Ni, Ramin Okhrati, Steven Guan, and Victor Chang. 2022. Knowledge Graph and Deep Learning-based Text-to-GraphQL Model for Intelligent Medical Consultation Chatbot. *Information Systems Frontiers* 26, 1 (jul 2022), 137–156. <https://doi.org/10.1007/s10796-022-10295-0>
- [27] Lior Okman, Nurit Gal-Oz, Yaron Gonen, Ehud Gudes, and Jenny Abramov. 2011. Security Issues in NoSQL Databases. In *Proceedings of the 2011 IEEE 10th International Conference on Trust, Security and Privacy in Computing and Communications (TRUSTCOM '11)*. IEEE Computer Society, USA, 541–547. <https://doi.org/10.1109/TrustCom.2011.70>
- [28] Fatma Özcan, Abdul Quamar, Jaydeep Sen, Chuan Lei, and Vasilis Efthymiou. 2020. State of the Art and Open Challenges in Natural Language Interfaces to Data. *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (2020). <https://api.semanticscholar.org/CorpusID:218881987>
- [29] Octavian Popescu, Irene Manotas, Ngoc Phuoc An Vo, Hangu Yeo, Elahe Khorashani, and Vadim Sheinin. 2022. Addressing Limitations of Encoder-Decoder Based Approach to Text-to-SQL. In *Proceedings of the 29th International Conference on Computational Linguistics*, Nicoletta Calzolari, Chu-Ren Huang, Hansaem Kim, James Pustejovsky, Leo Wanner, Key-Sun Choi, Pum-Mo Ryu, Hsin-Hsi Chen, Lucia Donatelli, Heng Ji, Sadao Kurohashi, Patrizia Paggio, Nianwen Xue, Seokhwan Kim, Younggyun Hahm, Zhong He, Tony Kyungil Lee, Enrico Santus, Francis Bond, and Seung-Hoon Na (Eds.). International Committee on Computational Linguistics, Gyeongju, Republic of Korea, 1593–1603. <https://aclanthology.org/2022.coling-1.137>
- [30] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 36339–36348. https://proceedings.neurips.cc/paper_files/paper/2023/file/72223cc66f63ca1aa59daec1b3670e6-Paper-Conference.pdf
- [31] Jieqing Qi, Jingyao Tang, Ziwei He, Xiangpeng Wan, Yu Cheng, Chenghu Zhou, Xinbing Wang, Quanshi Zhang, and Zhouhan Lin. 2022. RASAT: Integrating Relational Structures into Pretrained Seq2Seq Model for Text-to-SQL. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 3215–3229. <https://doi.org/10.18653/v1/2022.emnlp-main.211>
- [32] Abdul Quamar, Vasilis Efthymiou, Chuan Lei, and Fatma Özcan. 2022. Natural Language Interfaces to Data. *Foundations and Trends® in Databases* 11, 4 (2022), 319–414. <https://doi.org/10.1561/19000000078>

- [33] Tonghui Ren, Yuankai Fan, Zhenying He, Ren Huang, Jiaqi Dai, Can Huang, Yanan Jing, Kai Zhang, Yifan Yang, and X. Sean Wang. 2024. PURPLE: Making a Large Language Model a Better SQL Writer. *arXiv:2403.20014* [cs.DB] <https://arxiv.org/abs/2403.20014>
- [34] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- [35] Jaydeep Sen, Chuan Lei, Abdul Quamar, Fatma Özcan, Vasilis Efthymiou, Ayushi Dalmia, Greg Stager, Ashish Mittal, Diptikalyan Saha, and Karthik Sankaranarayanan. 2020. ATHENA++: natural language querying for complex nested SQL queries. *Proc. VLDB Endow.* 13, 12 (jul 2020), 2747–2759. <https://doi.org/10.14778/3407790.3407858>
- [36] Site24x7. 2023. SQL to MongoDB Query Converter. <https://www.site24x7.com/tools/sql-to-mongodb.html>. Accessed: 2025-1-11.
- [37] Yuanfeng Song, Raymond Chi-Wing Wong, and Xuefang Zhao. 2024. Speech-to-SQL: toward speech-driven SQL query generation from natural language question. *The VLDB Journal* (2024), 1–23.
- [38] Yuanfeng Song, Raymond Chi-Wing Wong, Xuefang Zhao, and Di Jiang. 2022. VoiceQuerySystem: A voice-driven database querying system using natural language questions. In *Proceedings of the 2022 International Conference on Management of Data*. 2385–2388.
- [39] Michael Staniek, Raphael Schumann, Maike Züfle, and Stefan Giezler. 2024. Text-to-OverpassQL: A Natural Language Interface for Complex Geodata Querying of OpenStreetMap. *Transactions of the Association for Computational Linguistics* 12 (2024), 562–575. https://doi.org/10.1162/tacl_a_00654
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 6000–6010.
- [41] Harley Vera-Olivera, Ruizhe Guo, Ruben Cruz Huacarpuma, Ana Paula Bernardi Da Silva, Ari Melo Mariano, and Maristela Holanda. 2021. Data modeling and nosql databases-a systematic mapping review. *ACM Computing Surveys (CSUR)* 54, 6 (2021), 1–26.
- [42] vincentrussell. 2024. SQL to MongoDB Query Converter. <https://github.com/vincentrussell/sql-to-mongo-db-query-converter.git>. Accessed: 2025-1-16.
- [43] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 7567–7578. <https://doi.org/10.18653/v1/2020.acl-main.677>
- [44] Lihan Wang, Bowen Qin, Binyuan Hui, Bowen Li, Min Yang, Bailin Wang, Binhua Li, Jian Sun, Fei Huang, Luo Si, and Yongbin Li. 2022. Proton: Probing Schema Linking Information from Pre-trained Language Models for Text-to-SQL Parsing. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Washington DC, USA) (KDD '22)*. Association for Computing Machinery, New York, NY, USA, 1889–1898. <https://doi.org/10.1145/3534678.3539305>
- [45] Kun Xu, Lingfei Wu, Zhiguo Wang, Yansong Feng, and Vadim Sheinin. 2018. SQL-to-Text Generation with Graph-to-Sequence Model. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 931–936. <https://doi.org/10.18653/v1/D18-1112>
- [46] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. <https://doi.org/10.18653/v1/D18-1425>
- [47] Albert Zeyer, Parnia Bahar, Kazuki Irie, Ralf Schlüter, and Hermann Ney. 2019. A comparison of Transformer and LSTM encoder decoder models for ASR. In *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 8–15.
- [48] Xiangyu Zhao, Longbiao Wang, Ruifang He, Ting Yang, Jinxin Chang, and Ruifang Wang. 2020. Multiple knowledge syncretic transformer for natural dialogue generation. In *Proceedings of The Web Conference 2020*. 752–762.
- [49] Yanzhao Zheng, Haibin Wang, Baohua Dong, Xingjun Wang, and Changshan Li. 2022. HIE-SQL: History Information Enhanced Network for Context-Dependent Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics: ACL 2022*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 2997–3007. <https://doi.org/10.18653/v1/2022.findings-acl.236>
- [50] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *arXiv:1709.00103* [cs.CL] <https://arxiv.org/abs/1709.00103>
- [51] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103* (2017).

8 ANALYSIS OF SQL-TO-NOSQL APPROACHES

Text-to-SQL, a classic task in the field of natural language processing, aims to automatically convert natural language queries into structured query language (SQL). It has a long research history and has accumulated a wealth of achievements [8, 30, 43, 46]. However, with the widespread adoption of NoSQL databases in various application scenarios, exploring how to extend Text-to-SQL technology to the NoSQL domain through a cascading approach is of significant importance. Specifically, this process can be divided into multiple stages: First, existing Text-to-SQL models (such as SQLNet, TypeSQL, RAT-SQL, etc.) are used to convert natural language queries into SQL queries. For example, given the natural language query "Find all users older than 30", the model generates the corresponding SQL query: "SELECT * FROM users WHERE age > 30". Subsequently, based on the syntactic mapping relationship between SQL and NoSQL, the generated SQL query is converted into the query language of the target NoSQL database. This process requires in-depth analysis of the syntax and semantics of both query languages. For instance, for the document-oriented database MongoDB, the SQL "SELECT" statement can be mapped to the "find" operation, and the "WHERE" condition can be mapped to a query filter, thereby transforming the aforementioned SQL query into a MongoDB query: "db.users.find({'age': {'\$gt': 30}})".

Currently, there are several websites specifically designed for SQL-to-NoSQL conversion based on the syntax mapping between SQL and NoSQL [16, 36], as well as GitHub projects [42]. We utilized an open-source project from GitHub to transform the test set of the TEND dataset (which includes NoSQL queries and their corresponding SQL queries), resulting in the SQL-to-NoSQL by Grammar presented in Table 5 (Grammar Converter) and Table 6. Table 5 displays the accuracy of the converted NoSQL queries, while Table 6 showcases some representative examples.

	Query-based		
Method	EM	QSM	QFC
SMART (Ours)	23.82%	63.21%	75.60%
Grammar Converter	0.00%	17.12%	23.24%
LLM Converter w. SQL Schema	10.09%	58.05%	64.90%
LLM Converter w/o. SQL Schema	7.71%	57.80%	63.35%
(a) Query-based Results			
	Execution-based		
Method	EX	EFM	EVM
SMART (Ours)	65.08%	87.21%	72.29%
Grammar Converter	10.81%	70.45%	67.57%
LLM Converter w. SQL Schema	44.76%	71.57%	73.55%
LLM Converter w/o. SQL Schema	47.68%	69.19%	73.80%

(b) Execution-based Results

Table 5: Experimental Results of SQL-to-NoSQL.

Easy Find Query	
SQL	SELECT Address FROM Restaurant WHERE ResName = "Subway";
Target NoSQL	db.Restaurant.find({ "ResName": "Subway" }, { "Address": 1, "_id": 0 });
Converted NoSQL	db.Restaurant.find({ "ResName": "Subway" }, { "Address": 1 });
Medium Find Query	
SQL	SELECT last_name FROM staff WHERE email_address LIKE "%wrau%"
Target NoSQL	db.Staff.find({ email_address: { \$regex: "wrau", \$options: "i" } }, { last_name: 1, _id: 0 });
Converted NoSQL	db.staff.find({ "email_address": { "\$regex": ".*wrau.*" } }, { "_id": 0, "last_name": 1 })
Hard Find Query	
SQL	SELECT T2.Fname , T2.Lname FROM COURSE AS T1 JOIN FACULTY AS T2 ON T1.Instructor = T2.FacID WHERE T1.CName = "COMPUTER LITERACY";
Target NoSQL	db.Faculty.find({ "Course.CName": "COMPUTER LITERACY" }, { "Fname": 1, "Lname": 1, "_id": 0 });
Converted NoSQL	db.COURSE.aggregate([{ "\$match": { "CName": "COMPUTER LITERACY" } }, { "\$lookup": { "from": "FACULTY", "let": { "instructor": "\$Instructor" }, "pipeline": [{ "\$match": { "\$expr": { "\$eq": ["\$\$instructor", "\$FacID"] } } }], "as": "T2" } }, { "\$unwind": { "path": "\$T2", "preserveNullAndEmptyArrays": false } }, { "\$project": { "_id": 0, "T2.Fname": 1, "T2.Lname": 1 } }])
Easy Aggregate Query	
SQL	SELECT Nationality FROM pilot GROUP BY Nationality ORDER BY COUNT(*) DESC LIMIT 1
Target NoSQL	db.pilot.aggregate([{ \$group: { _id: "\$Nationality", count: { \$sum: 1 } } }, { \$sort: { count: -1 } }, { \$limit: 1 }, { \$project: { _id: 0, Nationality: "\$_id" } }]);
Converted NoSQL	db.pilot.aggregate([{ "\$group": { "_id": "\$Nationality" } }, { "\$sort": { "count": -1 } }, { "\$limit": 1 }, { "\$project": { "Nationality": "\$_id", "_id": 0 } }])
Medium Aggregate Query	
SQL	SELECT company , main_industry FROM company WHERE company_id NOT IN (SELECT company_id FROM station_company)
Target NoSQL	db.company.aggregate([{ \$lookup: { from: "gas_station", localField: "Company_ID", foreignField: "station_company.Company_ID", as: "Docs1" } }, { \$match: { Docs1: { \$size: 0 } } }, { \$project: { Company: 1, Main_Industry: 1, _id: 0 } }]);
Converted NoSQL	Converting Error
Hard Aggregate Query	
SQL	SELECT T1.first_name , T1.last_name , T1.employee_id , T4.country_name FROM employees AS T1 JOIN departments AS T2 ON T1.department_id = T2.department_id JOIN locations AS T3 ON T2.location_id = T3.location_id JOIN countries AS T4 ON T3.country_id = T4.country_id
Target NoSQL	db.departments.aggregate([{ \$unwind: "\$employees" }, { \$lookup: { from: "regions", let: { location_id: "\$LOCATION_ID" }, pipeline: [{ \$unwind: "\$countries" }, { \$unwind: "\$countries.locations" }, { \$match: { \$expr: { \$eq: ["\$countries.locations.LOCATION_ID", "\$\$location_id"] } } }, { \$project: { COUNTRY_NAME: "\$countries.COUNTRY_NAME" } }], as: "Docs1" } }, { \$unwind: "\$Docs1" }, { \$project: { FIRST_NAME: "\$employees.FIRST_NAME", LAST_NAME: "\$employees.LAST_NAME", EMPLOYEE_ID: "\$employees.EMPLOYEE_ID", COUNTRY_NAME: "\$Docs1.COUNTRY_NAME", _id: 0 } }]);
Converted NoSQL	db.employees.aggregate([{ "\$lookup": { "from": "departments", "let": { "department_id": "\$department_id" }, "pipeline": [{ "\$match": { "\$expr": { "\$eq": ["\$\$department_id", "\$department_id"] } } }], "as": "T2" } }, { "\$unwind": { "path": "\$T2", "preserveNullAndEmptyArrays": false } }, { "\$lookup": { "from": "locations", "let": { "t2_location_id": "\$T2.location_id" }, "pipeline": [{ "\$match": { "\$expr": { "\$eq": ["\$\$t2_location_id", "\$location_id"] } } }], "as": "T3" }, { "\$unwind": { "path": "\$T3", "preserveNullAndEmptyArrays": false } }, { "\$lookup": { "from": "countries", "let": { "t3_country_id": "\$T3.country_id" }, "pipeline": [{ "\$match": { "\$expr": { "\$eq": ["\$\$t3_country_id", "\$country_id"] } } }], "as": "T4" }, { "\$unwind": { "path": "\$T4", "preserveNullAndEmptyArrays": false } }, { "\$project": { "_id": 0, "first_name": 1, "last_name": 1, "employee_id": 1, "T4.country_name": 1 } }])

Table 6: Samples of converting SQL to NoSQL through grammatical mapping relationships using SQL-to-NoSQL converter [42]. (Errors are marked with red colors)

From the experimental results presented in Table 5, we have observed that direct conversion based on grammatical rules is not feasible. This is fundamentally because such an approach entirely overlooks the significant differences between relational databases and NoSQL databases, culminating in a mere 10.09% accuracy rate on the most crucial metric — EX. This level of performance is far from sufficient to meet the demands of practical applications. Concurrently, we have analyzed the error samples from the Grammar Converter, with several representative examples showcased in Table 6. Based on the displayed examples, it is evident that the method of directly converting SQL queries into NoSQL queries according to grammatical mapping relationships can only be correct when the converted NoSQL query is a simple “find” query. However, this method becomes unfeasible when the NoSQL query to be converted becomes complex. Currently, large language models (LLMs) are revolutionizing and optimizing solutions across various research domains. The SQL-to-NoSQL conversion process might benefit from leveraging LLMs’ understanding of both programming languages. We attempted to provide an LLM (such as Deepseek-v3) with ground truth SQL queries and a MongoDB database schema,

instructing it to translate the SQL queries into corresponding MongoDB queries. The experimental setup was as follows: (i) Providing an SQL query, the complete database schema corresponding to the SQL query, and the complete MongoDB database schema (LLM Converter w. SQL Schema); (ii) Not providing the SQL database schema (LLM Converter w/o. SQL Schema). The experimental results obtained are also presented in Table 5.

Based on the results presented in Table 5, we have found that leveraging LLMs for SQL-to-NoSQL conversion demonstrates a certain level of feasibility. By furnishing the LLM with precise SQL queries, the corresponding database schema for the SQL queries, and the NoSQL database schema, we achieved an execution accuracy of 44.76%. This figure significantly surpasses the 10.81% accuracy rate attained by the Grammar Converter. However, it is important to note that in practical applications, obtaining accurate SQL queries is not always accurate. Incorrect SQL queries may lead to error accumulation, potentially resulting in a significant decline in accuracy. Therefore, to pursue higher accuracy, it may be necessary to design a more complex LLM framework, which represents a promising direction for further research.